Macaroons provide a solution to this problem by allowing anybody to append *caveats* to a capability that restrict how it can be used. Macaroons were invented by a team of academic and Google researchers in a paper published in 2014 (https://ai.google/research/pubs/pub41892).

> **DEFINITION** A ***macaroon*** is a type of cryptographic token that can be used to represent capabilities and other authorization grants. Anybody can append new ***caveats*** to a macaroon that restrict how it can be used.

To address our example, the user could append the following caveats to their capability to create a new capability that allows only read access to messages since lunchtime yesterday:

```
method = GET
since >= 2019-10-12T12:00:00Z
```

Macaroons use HMAC-SHA256 tags to protect the integrity of the token and any caveats just like the `HmacTokenStore` you developed in chapter 5. To allow anybody to append caveats to a macaroon, even if they don't have the key, macaroons make use of an interesting property of HMAC, which is that the authentication tag output from HMAC is indistinguishable from a random sequence of bytes and so can itself be used as a secret key.

To append a caveat to a macaroon you use the old authentication tag as the key to compute a new HMAC-